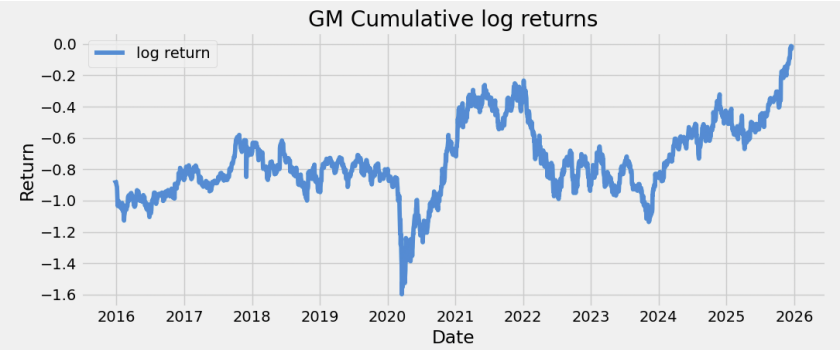
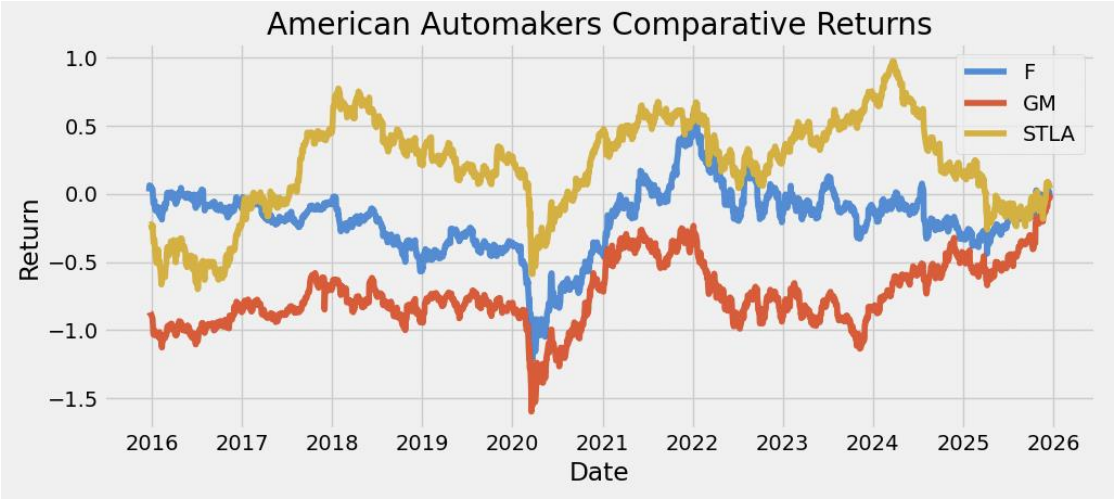
Stock Market Analysis

Progress Saturday Dec 27, 2025

- Made stock class for viewing stock performance over time



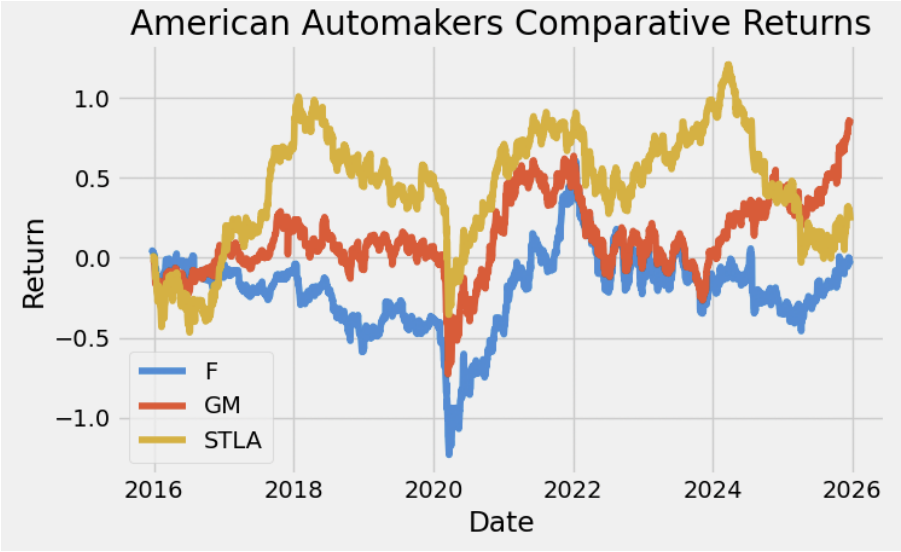
- Portfolio class for comparing performance over time



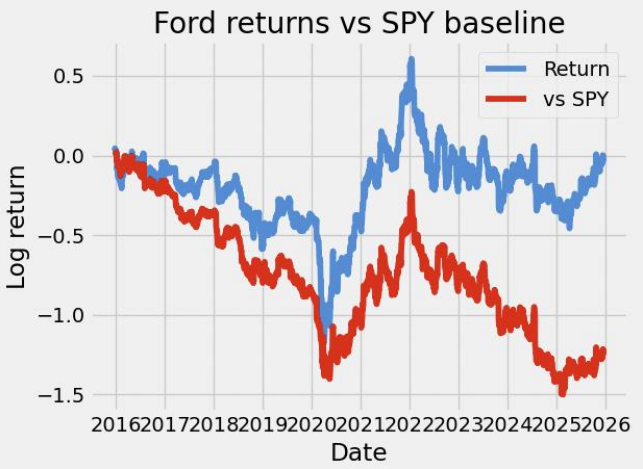
NEXT: Portfolio should show matching returns in beginning but not the end. This shows the opposite.

12/27/2025 stream 2

- Fixed plot



- Got log return vs a generic baseline. Works for Ford



- Fails for STLA & GM

STLA

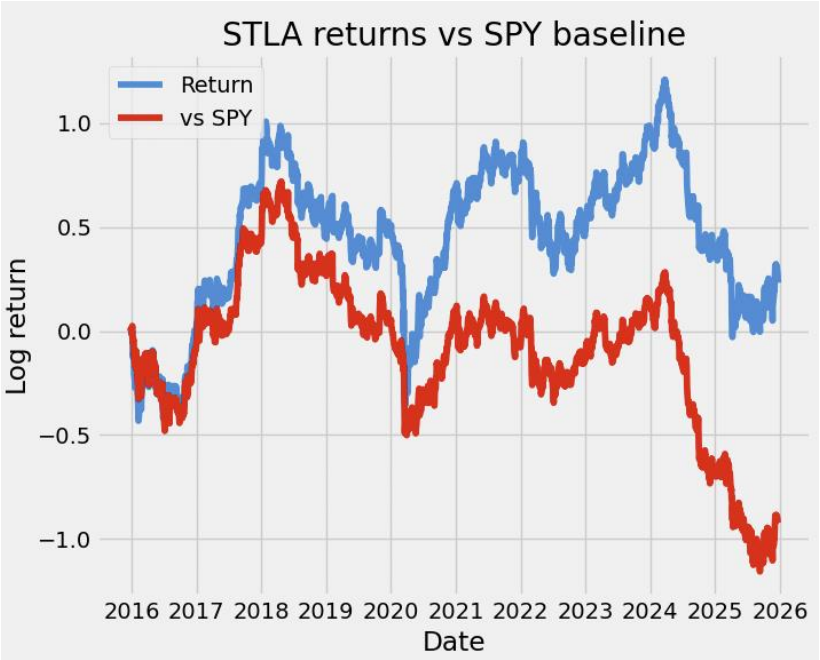
```
Traceback (most recent call last):
  File "/Users/wicks3/Desktop/Work/SPY/tests.py", line 115, in <module>
    comparative_return = stock.compare_to_baseline(baseline)
                         ^^^
```

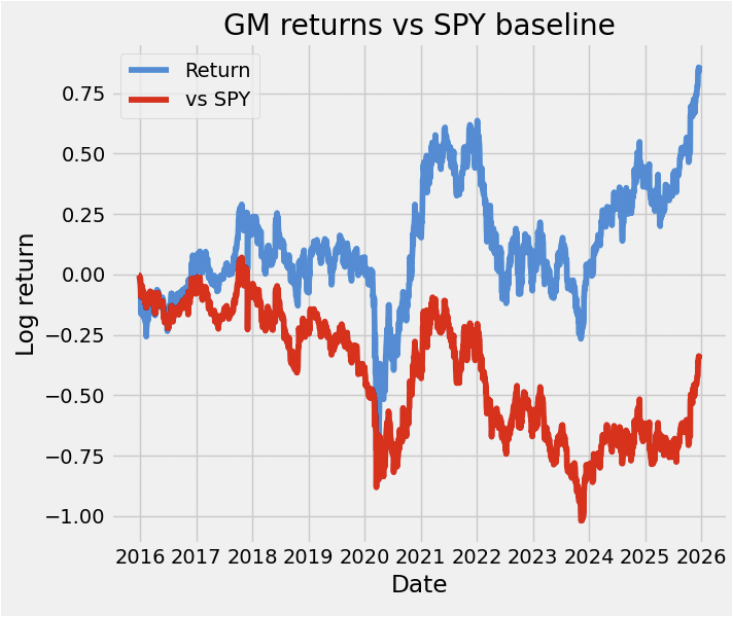
12/28/2025 – Getting STLA to work with SPY baseline

```
Traceback (most recent call last):
  File "/Users/wicks3/Desktop/Work/SPY/tests.py", line 115, in <module>
    comparative_return = stock.compare_to_baseline(baseline)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/wicks3/Desktop/Work/SPY/Analysis.py", line 58, in compare_to_baseline
    raise Exception("Unsolved error in compare_to_baseline")
Exception: Unsolved error in compare_to_baseline
(base) wicks3@epsspaceblack SPY %
```

Below: fixes problem w/ GM as well

Right: got it working

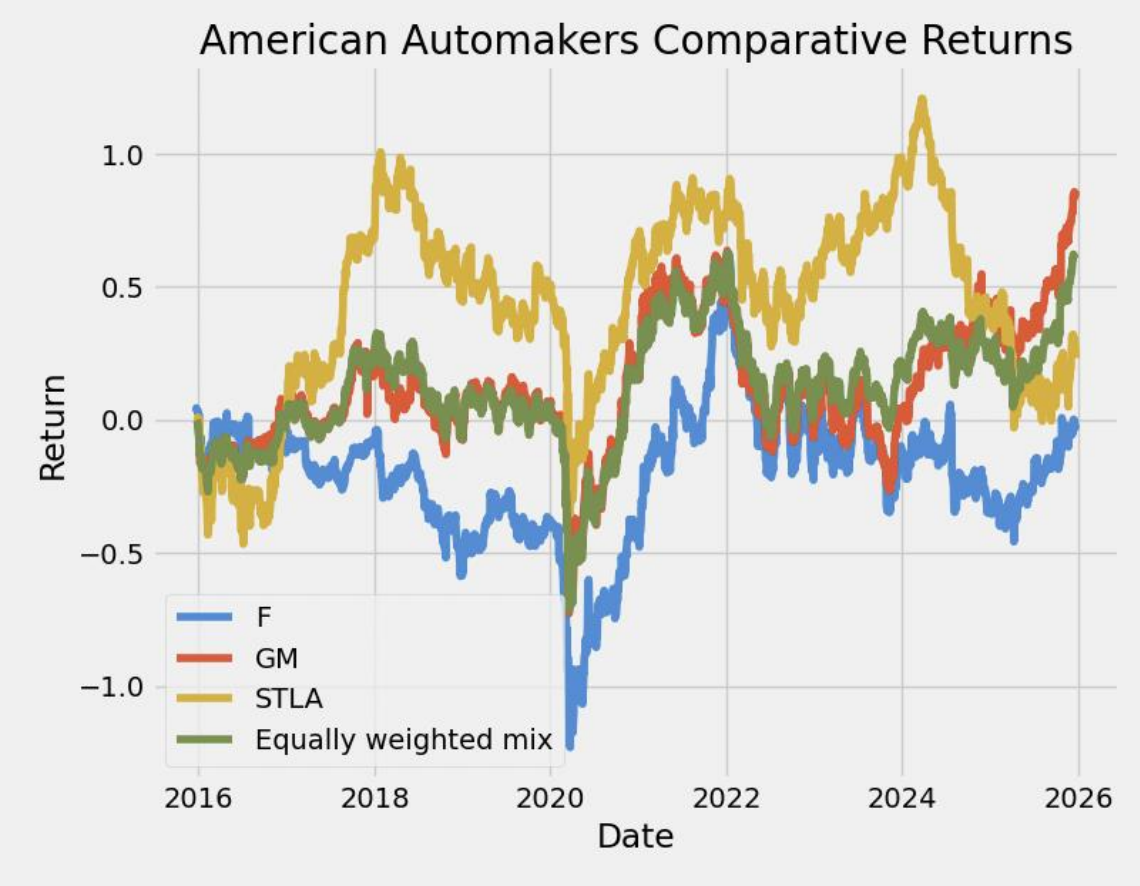




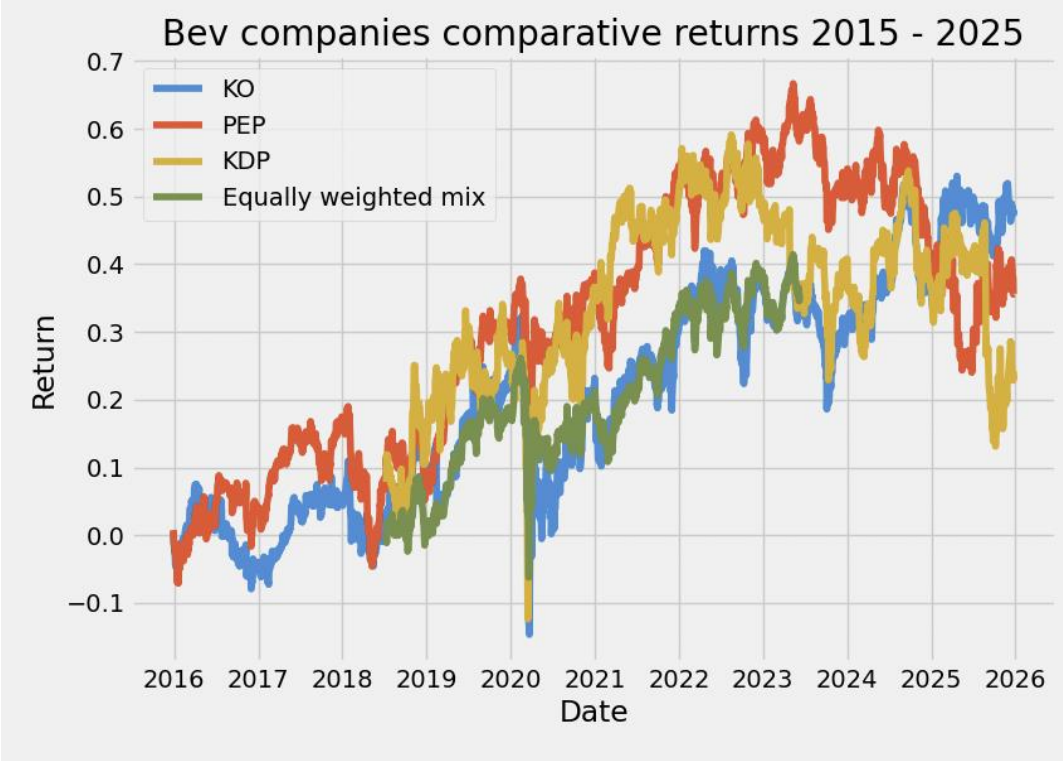
12/28/2025 progress

NEXT: ensure mix creation works properly for any setup
THEN: look into strategy implementation or sector analyses

- Got portfolio mix to work

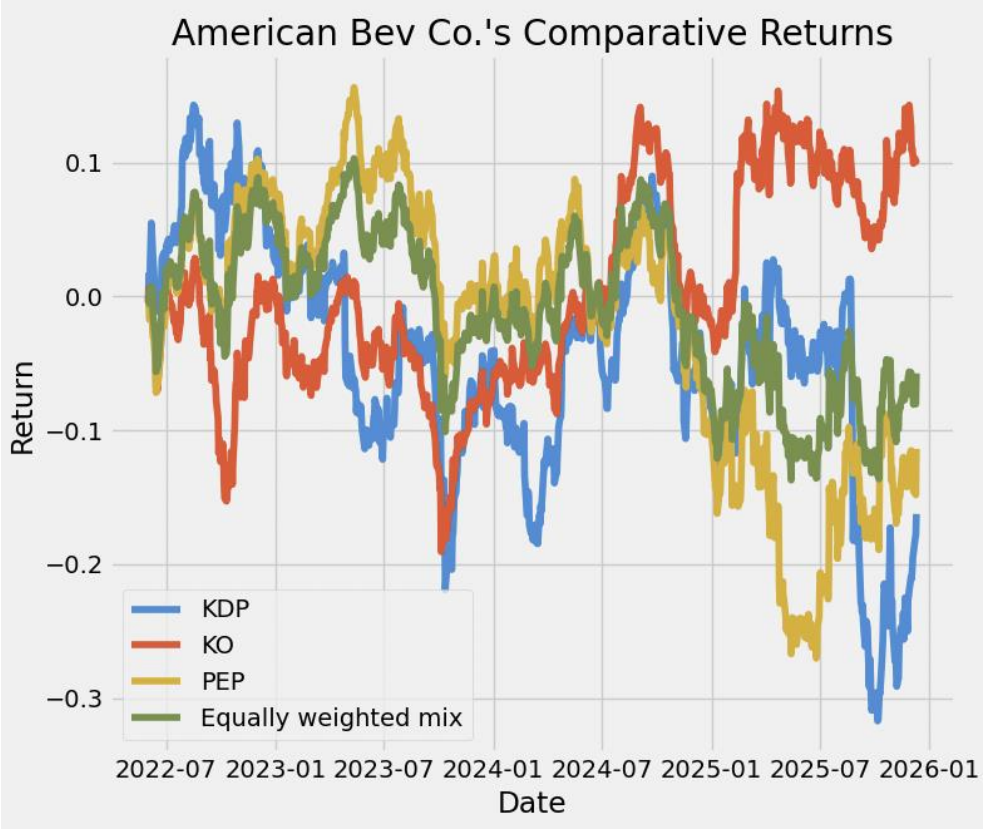


- Why is the data cut off towards the end for the beverage mix?



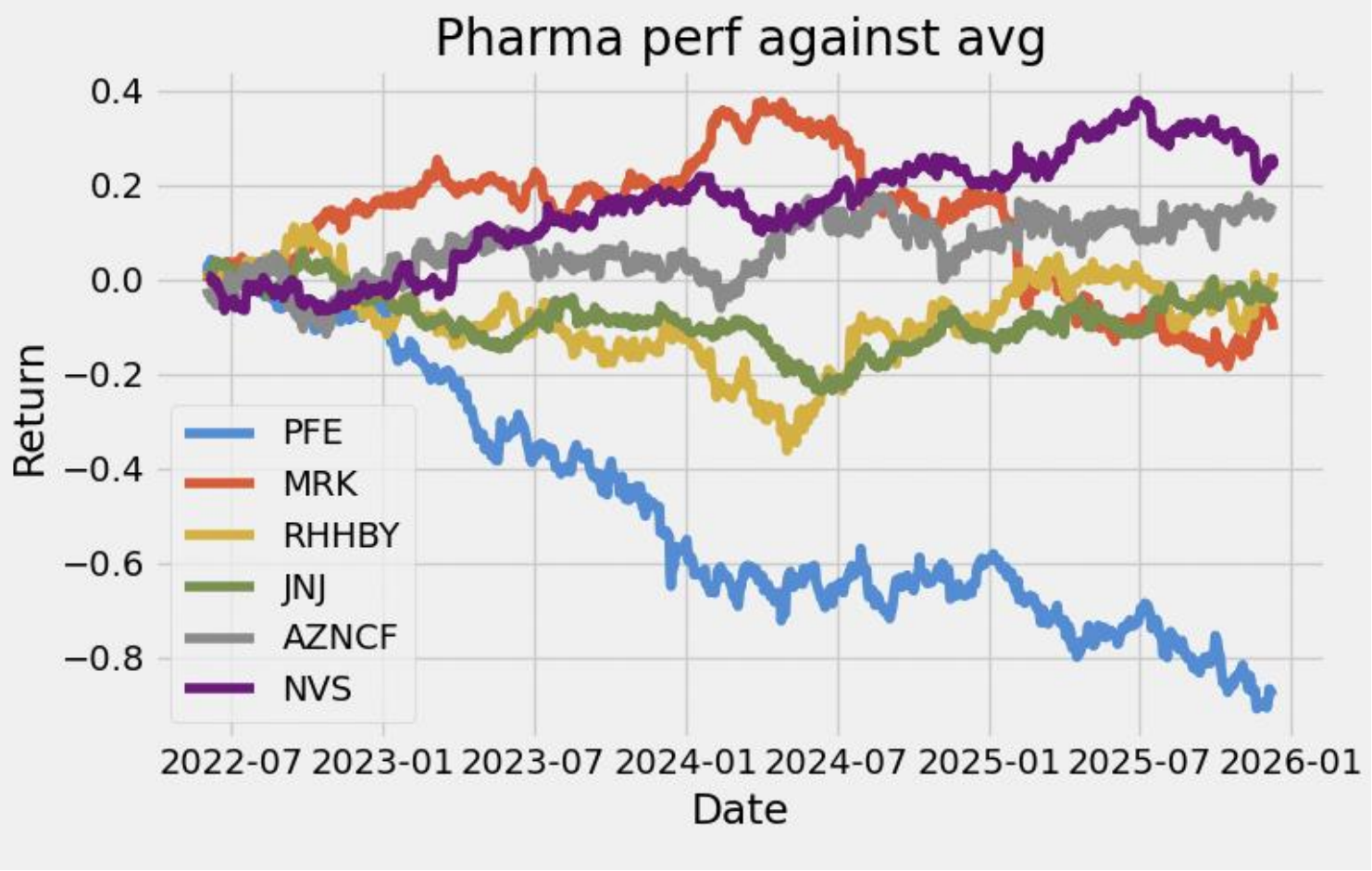
12/29/2025

Below: Fixed alignment for bev companies. Should work for everything now



1/1/2026

- Fixed data issue that stopped Pharmaceutical comparison from showing



NEXT: Finish getting fundamental data for analysis and integrate z-scores. Fundamental data source: https://www.dolthub.com/repositories/post-no-preference/earnings/data/master/balance_sheet_equity

1/2/2026

```
EV = Market Capitalization + Total Debt + Preferred Stock + Minority Interest – Cash and Cash Equivalents
```

```
EBITDA = Operating Income + Depreciation Expense + Amortization Expense
```

- Worked on implementing calculations for Z-score

z-score: use equally weighted z-score

- ROIC
- operating margin
- FCF margin
- EV/EBITDA (inverted)
- FCF yield
- Revenue CAGR
- EPS CAGR

ROIC
Operating Margin
FCF margin
EV/EBITDA
FCF Yield
Revenue CAGR
EPS CAGR

NEXT: Calculate all necessary variables for z-score calc

1/3/2026

- Datasets downloaded from site are all different shapes:

```
(Pdb) income_statement.shape
(237654, 24)
(Pdb) cash_flow_statement.shape
(127006, 25)
(Pdb) balance_sheet_assets.shape
(251451, 16)
(Pdb)
```

So I need to either reconcile the shapes or switch to using API instead of downloaded data

- Integrated earnings dataset from dolthub
- Wrote script to retrieve and visualize data obtained from earnings sql

```
         date act_symbol   period  ...  intangibles  deposits_and_other_assets  total_asse
ts
0  2012-12-31          F     Year  ...   87000000.0               5.000000e+09  1.905540e+
11
1  2013-12-31          F     Year  ...          0.0               5.847000e+09  2.020260e+
11
2  2014-12-31          F     Year  ...          0.0               6.353000e+09  2.085270e+
11
3  2015-12-31          F     Year  ...          0.0               7.466000e+09  2.249250e+
11
4  2016-09-30          F  Quarter  ...          0.0               7.111000e+09  2.349630e+
11
```

NEXT:
Integrate other datasets
Write methods for pulling relevant data and calculating z scores

1/4/2026

- Started constructing classes for reading data from Dolt
- Added other dolt DBs (options & stock prices)

NEXT:
Complete class for data reading & manipulation
Add functions for computing z-scores

Reading DOLT database

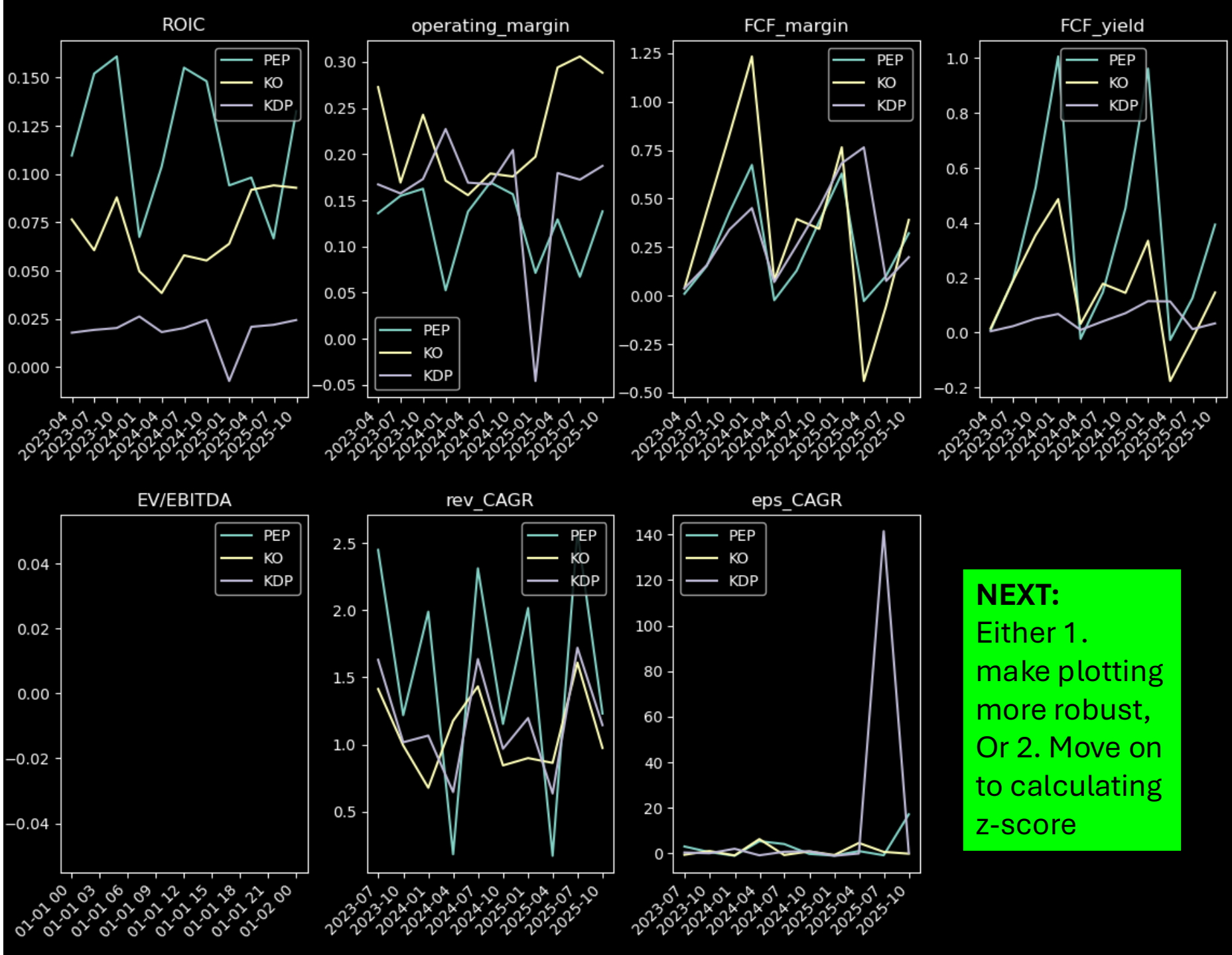
- 1/5/2026
  - Modified query to isolate data by stock name, date and report typ (quarterly or yearly)
    - Not all tables have Quarter/Year specification and some are predictive of future
    - **NEXT:** formulate strategy for reading data from these diverse datasets
- 1/6/2026
  - ROIC = NOPAT/Invested capital
  - Invested capital = total current assets - total current liabilities
  - NOPAT = pretax income – income taxes – non-operating income
  - Calculated ROIC
- 1/7/2026
  - Calculated operating margin

    Issue: incomplete data in cash_flow_statement causes errors when trying to calculate quantities that use free cash flow
  - Partway though calculating EV
- 1/8/2026
  - Dolthub db has no quarterly reports prior to 2022
  - Free cash flow margin = (operating cash flow - capex)/revenue
- 1/9/2026
  - Got rest of interesting fundamental variables

NEXT:
Make plotting for all variables with select stocks specified

1/14/2026

- Got fundamentals plot to work, but there are problems
  - 1. No EV/EBITDA data
  - 2. If we go before the beginning of 2023, it breaks due to some data problem



NEXT:
Either 1. make plotting more robust, Or 2. Move on to calculating z-score

Stocks for which fundamental plotting breaks

- STLA

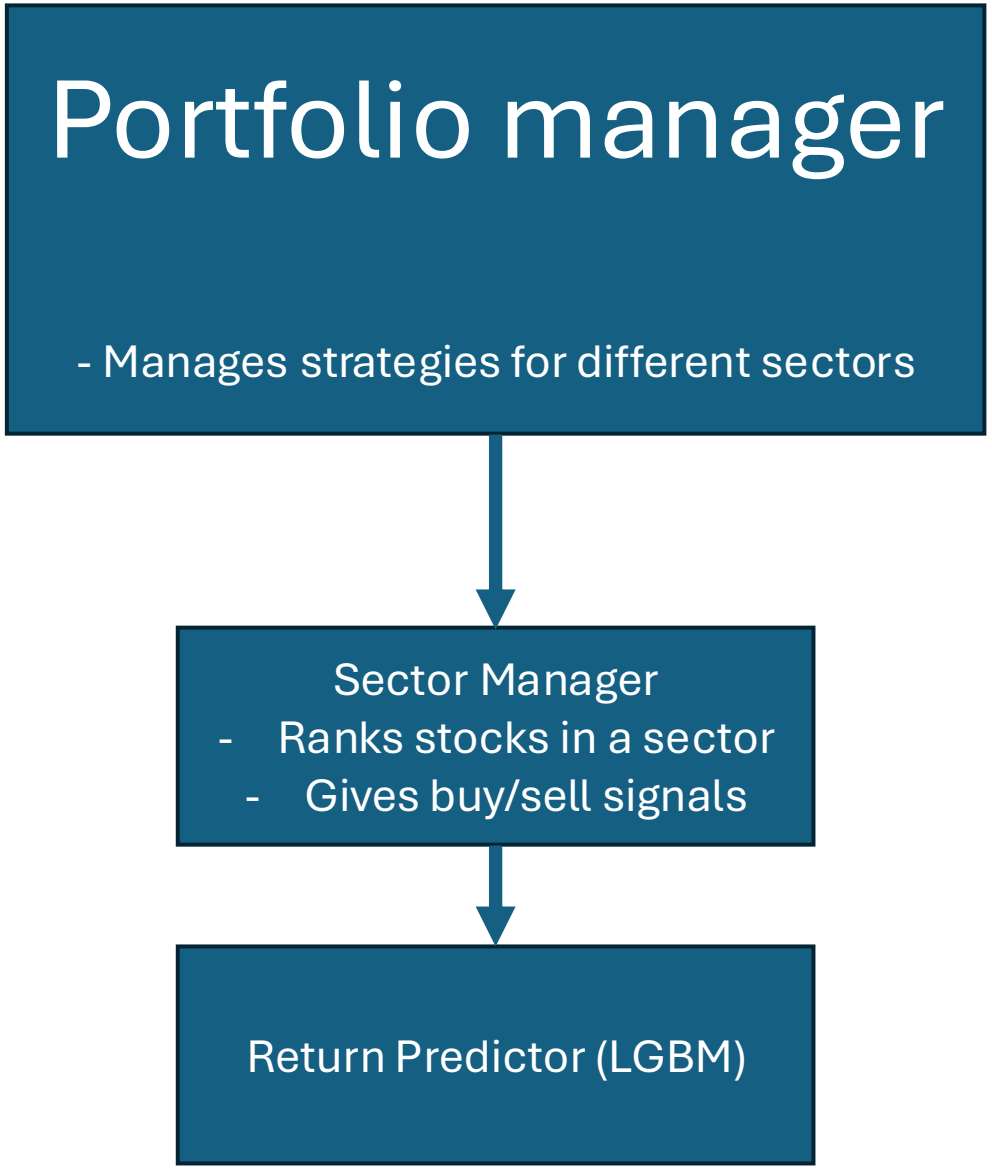
1/16/2026

- Figured out how to interface different databases simultaneously
- Started to read in dolt data for stocks, but encountering error.
  - Need to make more efficient (by not reading in data until we need it)
  - Proof of concept: specify dolt_data and make the same plots I made w/ NASDAQ data
    - If this doesn’t work I just need to build a new class for the dolt data

1/17/2026

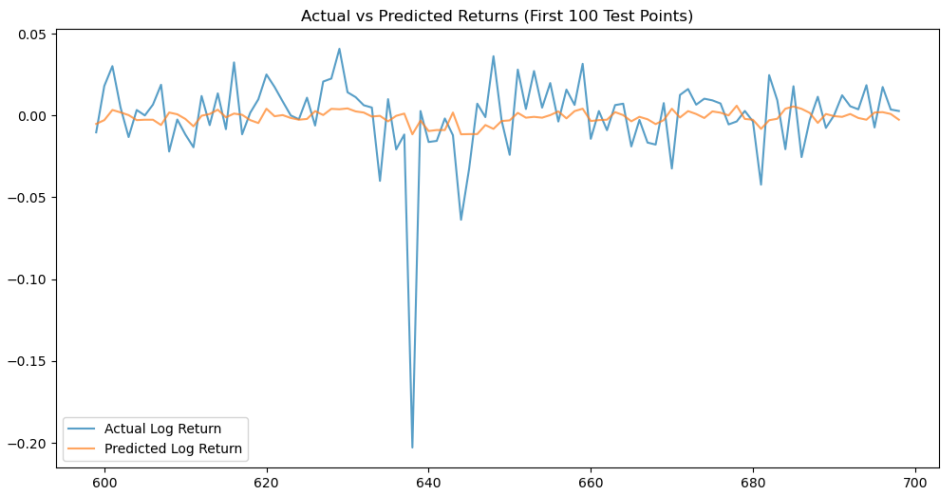
- How do I want to implement strategy?

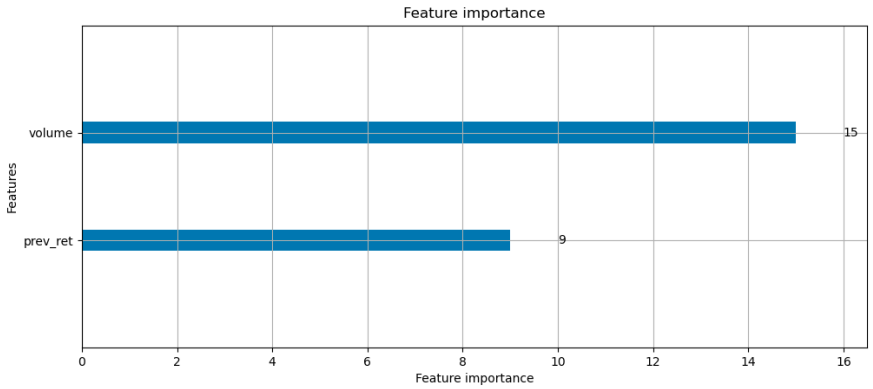
1. Define different stock universes for different sectors
2. Weight stocks in sector according to model specific to that universe
3. Use portfolio manager to weight different sectors
4. Assess performance on different sectors and each universe to cut losers and increase position on winners
5. Set up an environment to test this for a month


Portfolio manager
- Manages strategies for different sectors
Sector Manager
- Ranks stocks in a sector
- Gives buy/sell signals
Return Predictor (LGBM)

1/18/2026

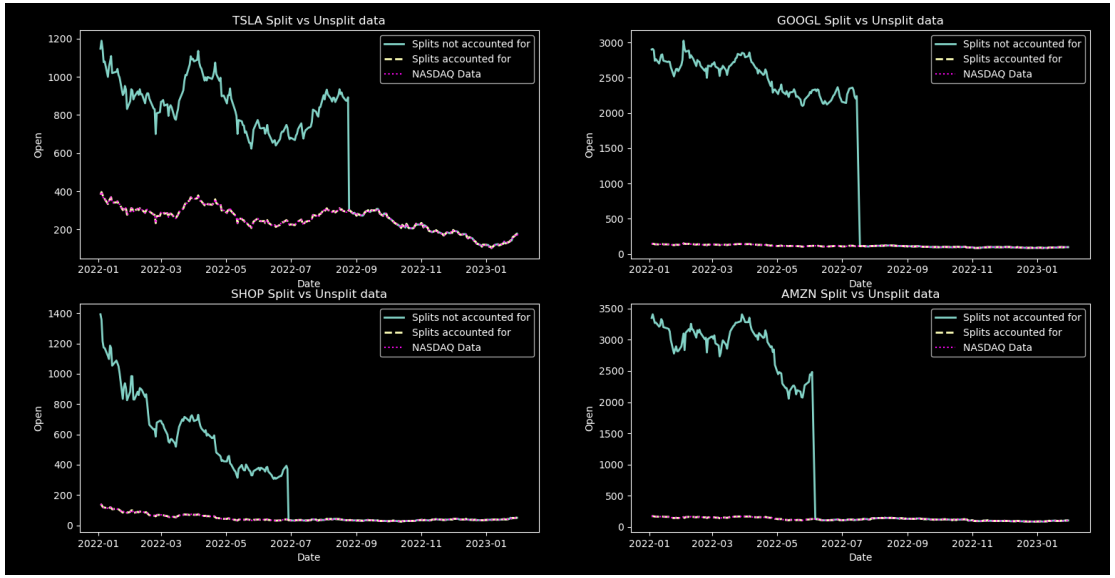
- Added model in simple_lgbm_run.py
  - Works poorly
  - Mixing quarterly report with price data is ugle
  - Concentrate on two strategies next:
    - Quarterly report data to predict quarterly return
    - Price data to predict log return
    - Do longer periods
    - See what works





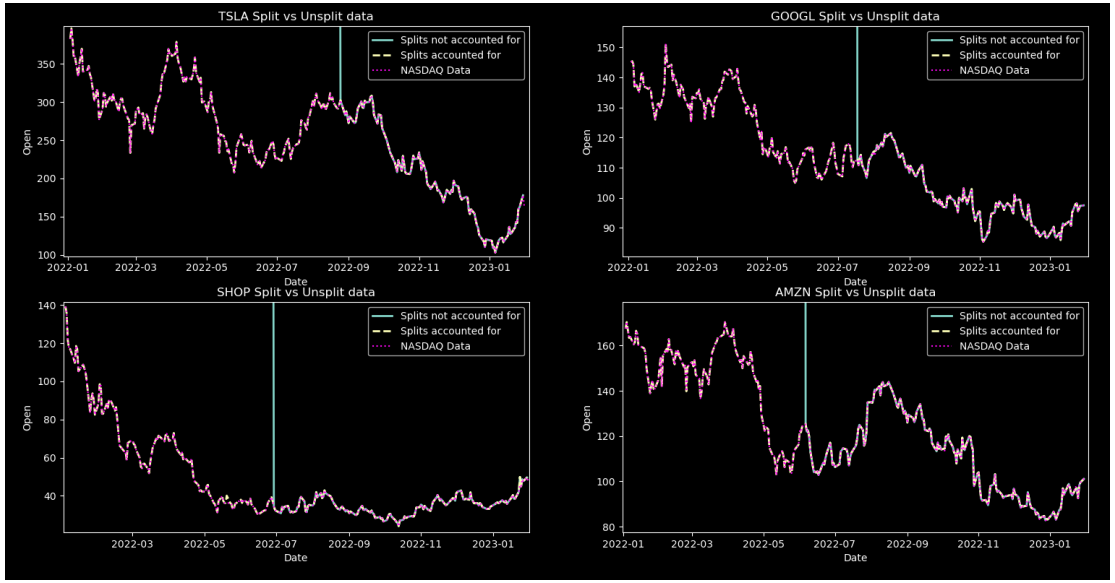
Beginning inputs suggested by Gemini: **Recommendation:** Start with just **Returns (1d, 5d, 20d)** and **Volatility (STD 20d)**. These usually contain 80% of the signal in price-only models. Add the others one by one to see if they actually improve the model.

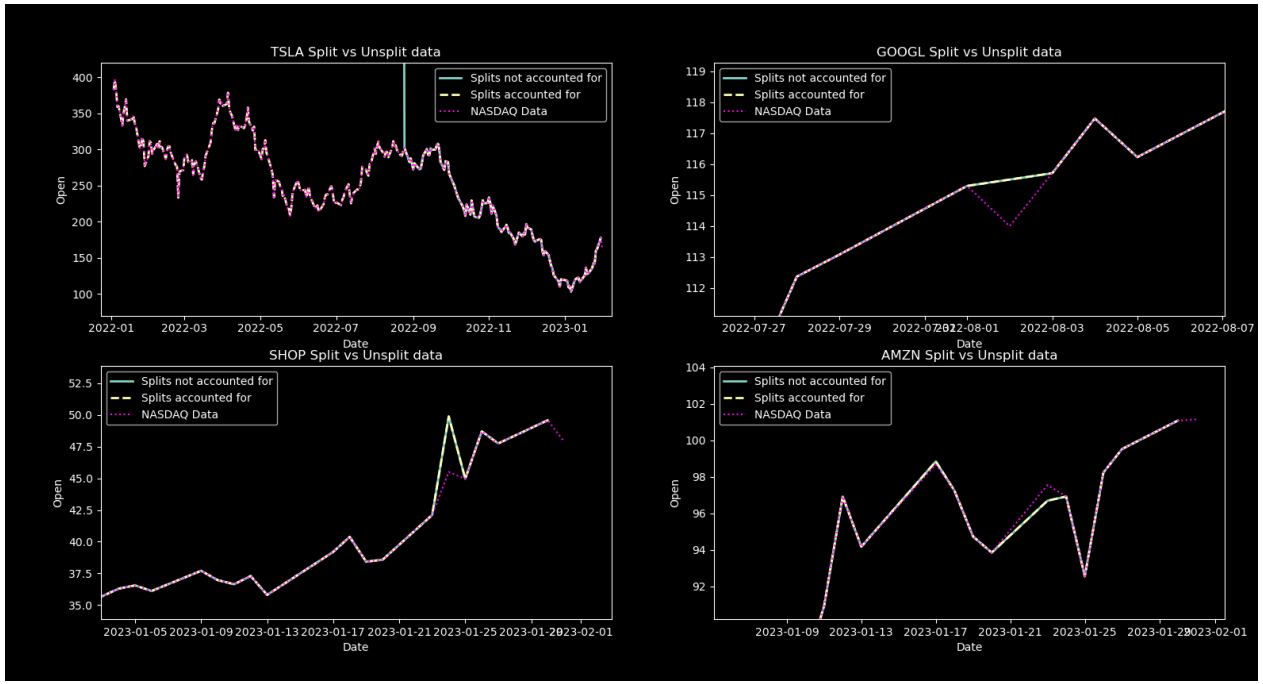
1/19/2026



NEXT:
Figure out discrepancies or decide if this is sufficient and move forward to building model

Top left: Stock split accounted for
Bottom left: Zoomed in to show before/after split
Below: There are some discrepancies between the data from post-no-preference and NASDAQ data





1/20/2026

- Did analysis on IBIT holdings
  - $42.50 price target for EOY
  - Purchased $45 put to expire June 2027 (above price target with long horizon so I can sell)
- Worked on inputs, added buy strategy
  - Leave buy strat for later
  - Focus on getting a good returns model

1/21/2026

- We have a model which works on a single stock
- I now want to train this model on *all* stocks which meet selection criteria (excluding those which are representative of bad data like noisy stocks and penny stocks)
- Before I can do this, I get the candidate stocks for each month. I need to add and trim stocks from the universe as necessary
- Once I have “good” data, I can train on the selected stocks
- Working on selection in stock_finder.py
  - Feed this to the analysis to train/test model

1/21/2026 (cont)

- Next: finish stock_finder.py
  - Filter out ETF data before getting volume
- I want to test:
  - What works best for the returns model when picking a universe of stocks?
    - Top 1000 volume
      - Performs worse than 50%

```
--- RESULTS ---
RMSE: 0.023779
Directional Accuracy: 49.41%
```

    - Volatility filter + hurst exponent filter
      - Performs slightly better than 50%

```
--- RESULTS (Hurst Universe) ---
RMSE: 0.042795
Directional Accuracy: 50.56%
```

    - Volatility filter + hurst filter + dickey-fuller filter

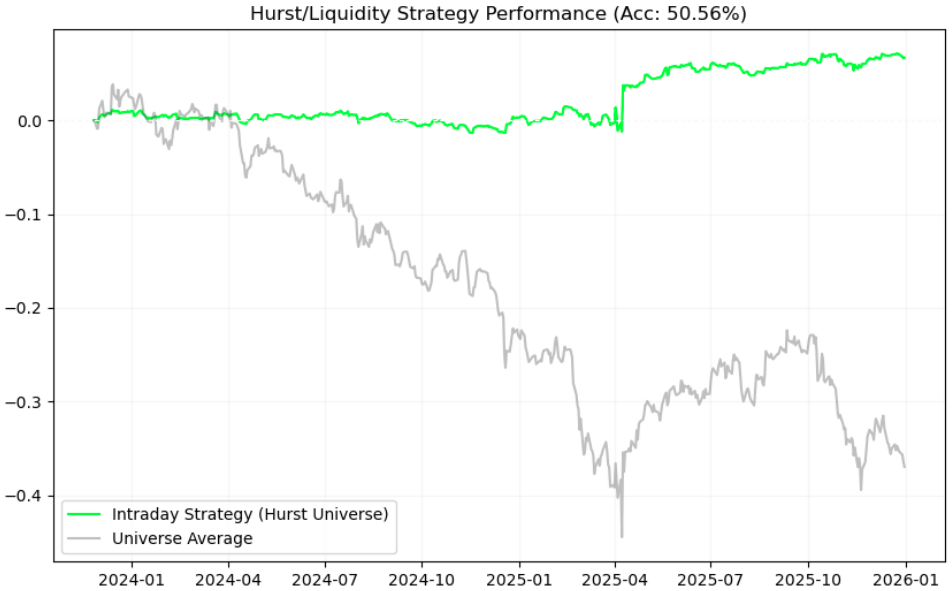
1/22/2026

NEXT:
Use universe data to train model
Then assess different methods of retrieving universe data
Put universe data retrieval into a single function/class

- Finished writing script to get universe data

1/23/2026

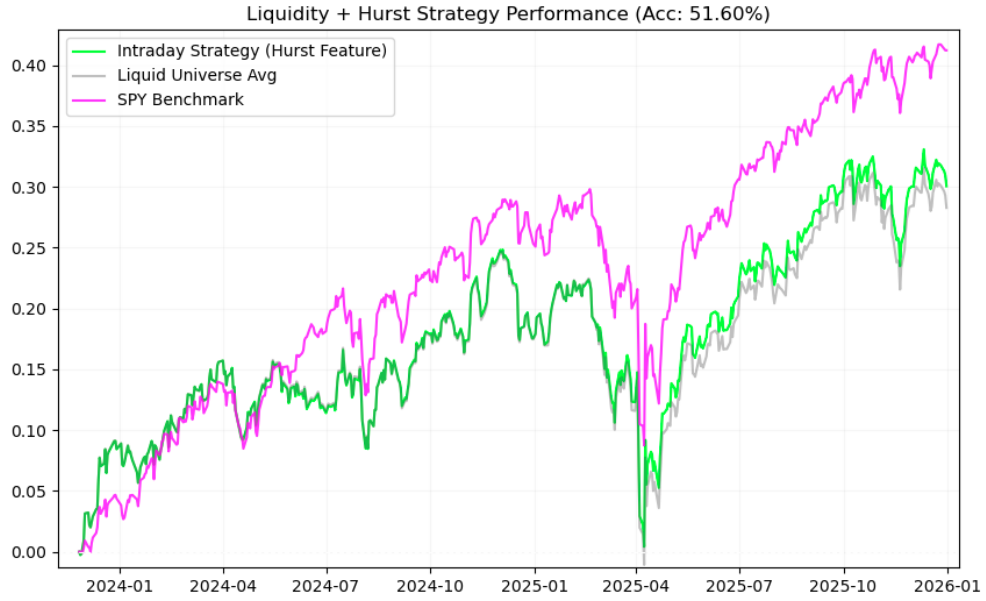
- Result from volume and hurst outperforms average, but the average is much worse than the S&P 500, why?
- It is consistent in returns
- NEXT: Dig into model and ticker selection to figure out what is going on

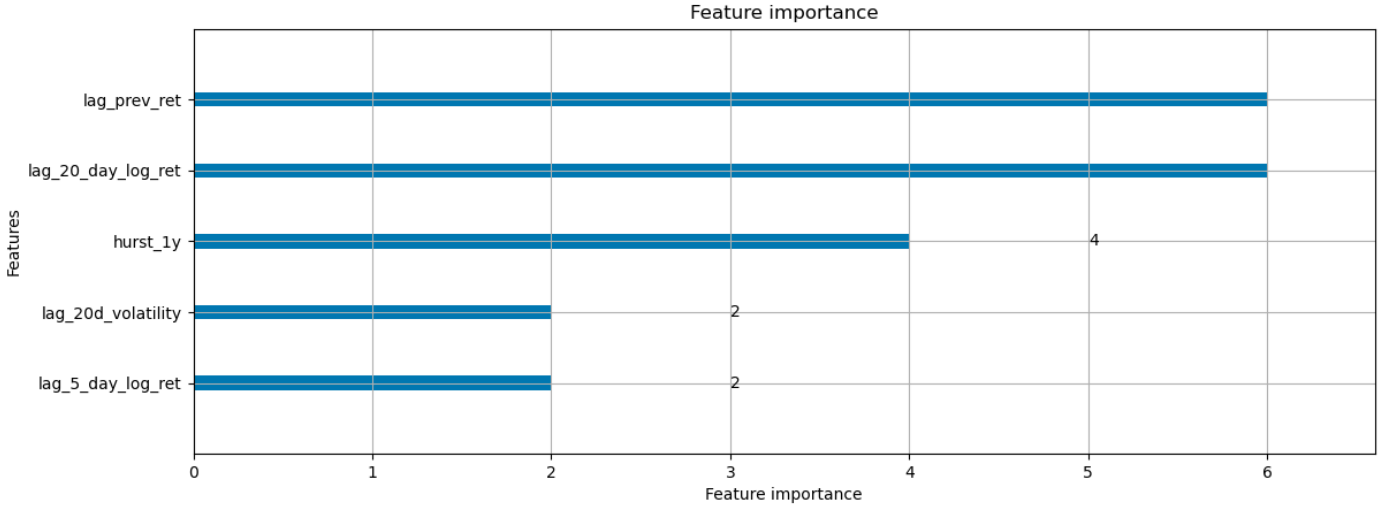


1/25/2026

NEXT:
Fix dolt databases so that they all activate at once.

- Got good returns model
- Next: build portfolio layer to weight buys and sells



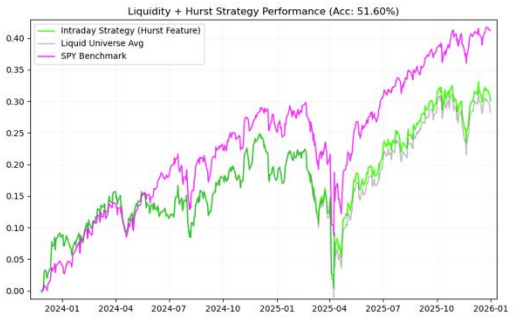


Above – Why does sigmoid strategy perform worse than picking all?

1/26/2026

Does changing the universe help?
- Add filter to pick lowest 500 by volatility after filtering top 1000 by liquidity

- Returns model works

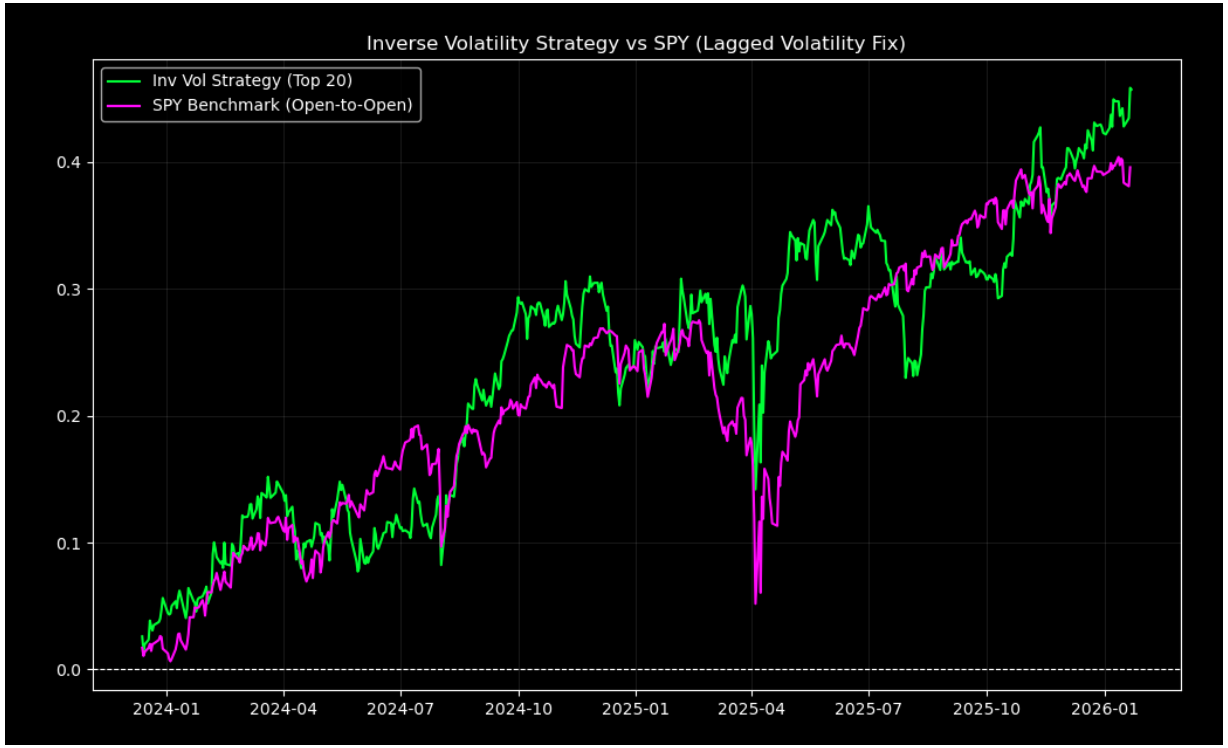


- But portfolio model fails

- Works with changes
  - Predicting return over avg instead of return
  - Inverse volatility weighting instead of sigmoid weighting

1/26/2026 continued …

- Fixed leakage and here is the current result:



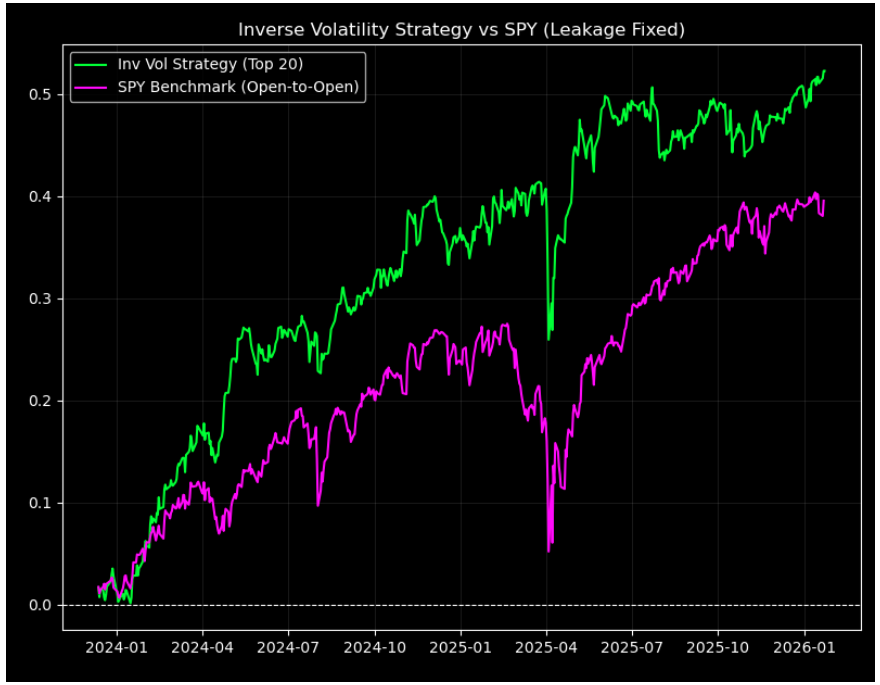
NEXT:

We have returns going back to 2022. Current model trains on price data going back to 2011. If we add in fundamental data, we get better metrics but have to sacrifice a lot of training data. **Does this improve the model?**

- Suggestions for improvement:
  - Use cross sectional ranking
  - Scale inputs by volatility
  - Change objective function from RMSE to MAE or Huber loss
  - Target winsorization (remove extreme outliers)
  - Add overnight return vs yesterday's intraday return
  - Use walk-forward validation

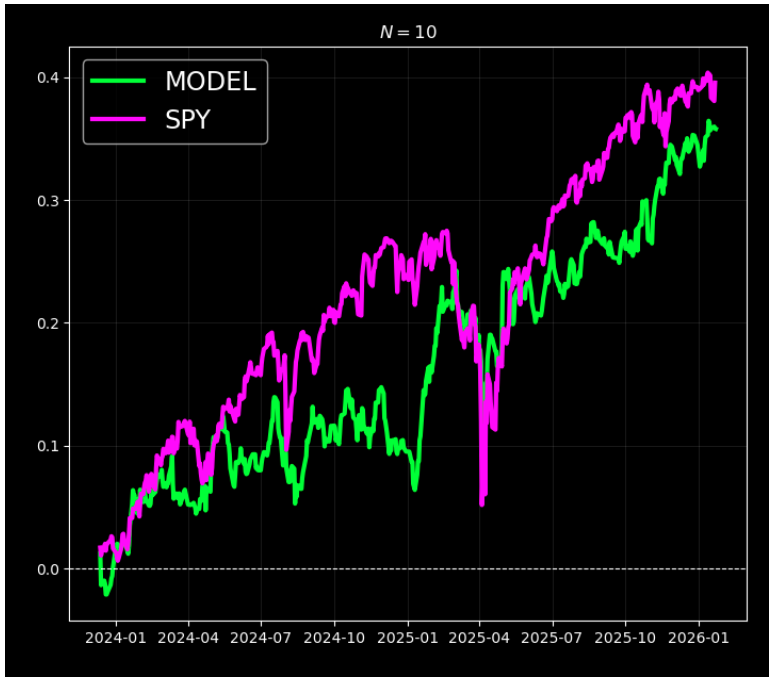
1/26/2025 continued

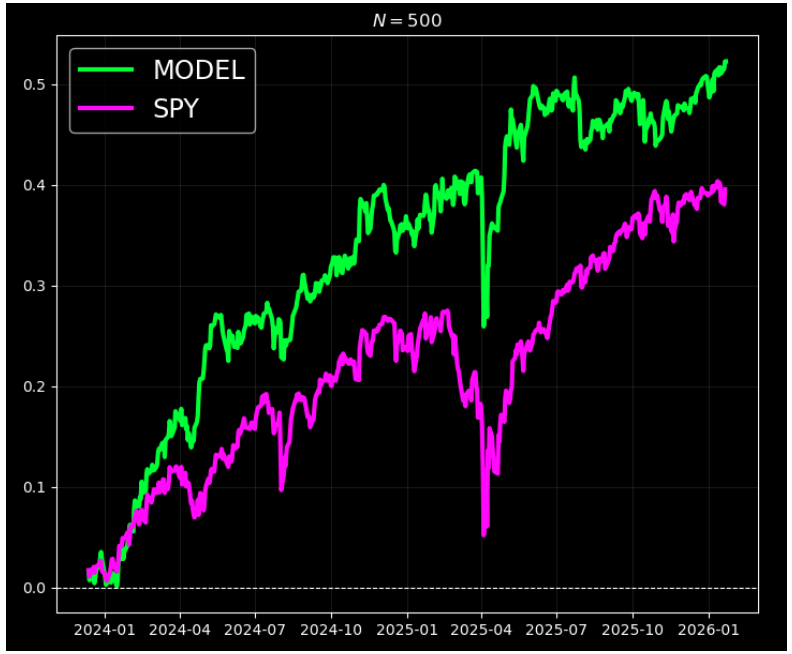
- Test works well if we train on the large universe (filtering top 1000 by liquidity, then lowest 500 by volatility, attained in get_data_v3.py) and then test on small universe (same as v3 data but only gets lowest 50 by volatility, attained in get_data_v4.py)

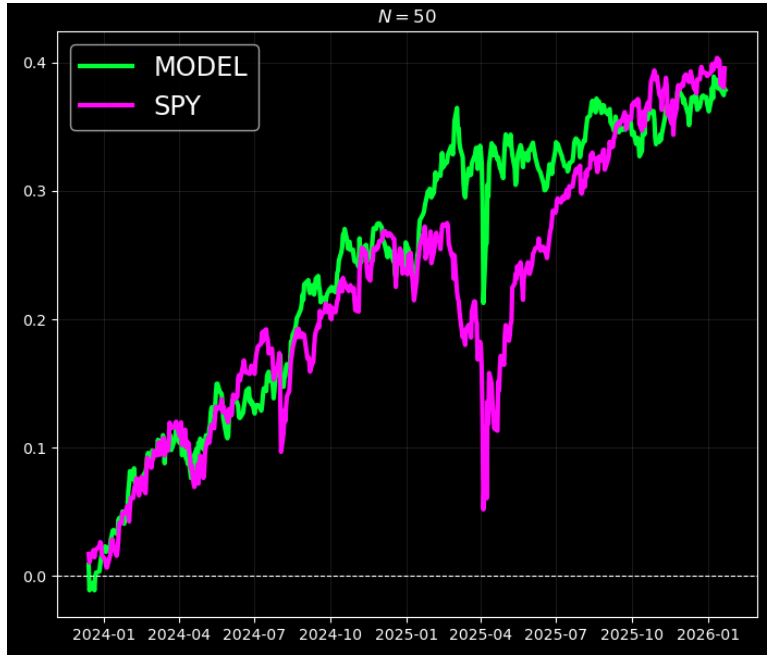


1/27/2026

- Tightening Universe filter lowers test performance
- Filter method:
  1. Get 1000 highest liquidity stocks for past month
  2. Filter out stocks which have dropped > 60% in the last year
  3. filter out stocks worth less than $15
  4. select the lowest N volatility stocks remaining

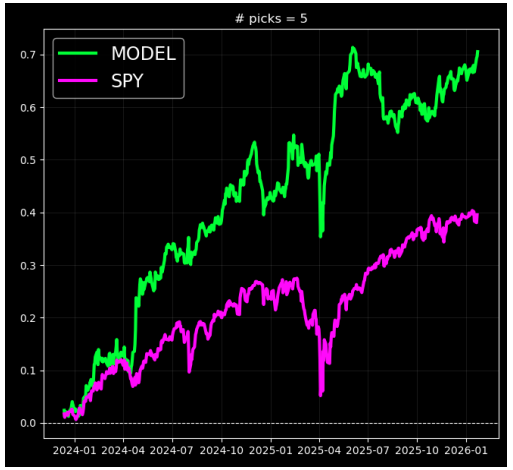


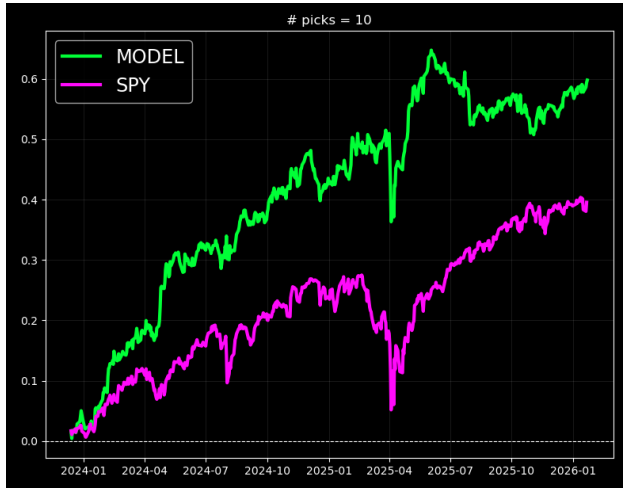


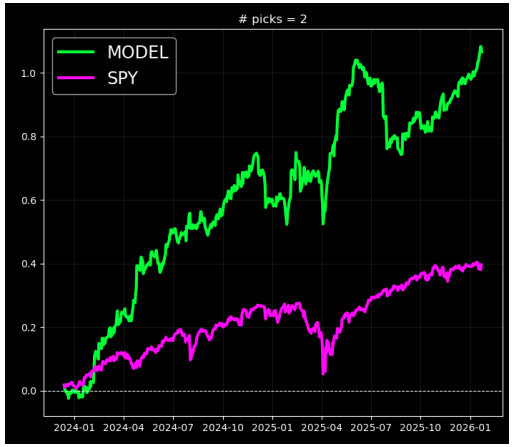


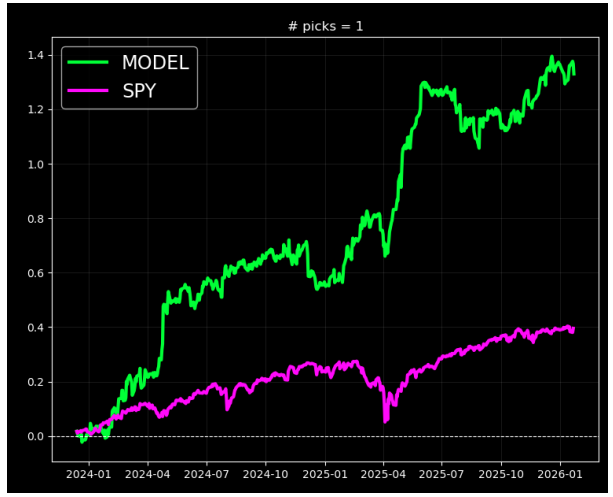
1/27/2026 (cont)

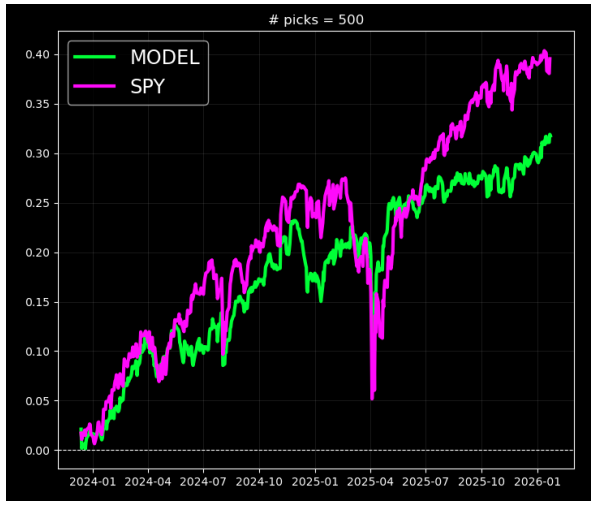
- Using entire training universe with a lower # of picks in the test performs better

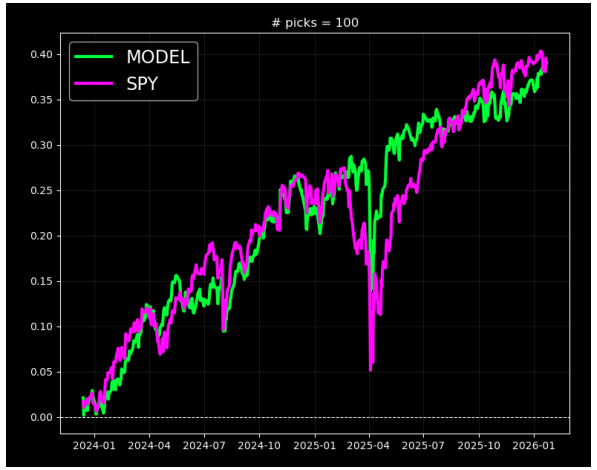


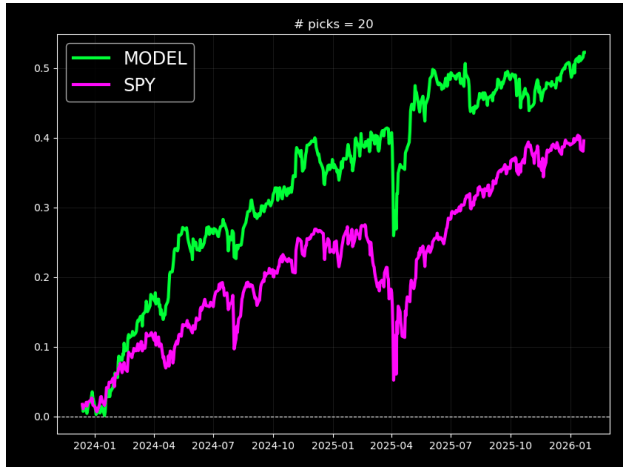


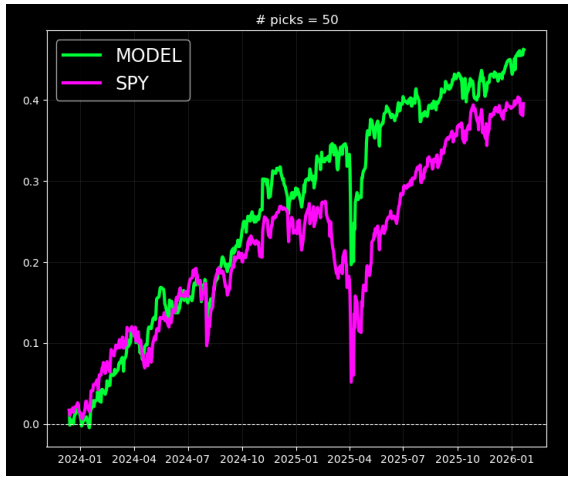






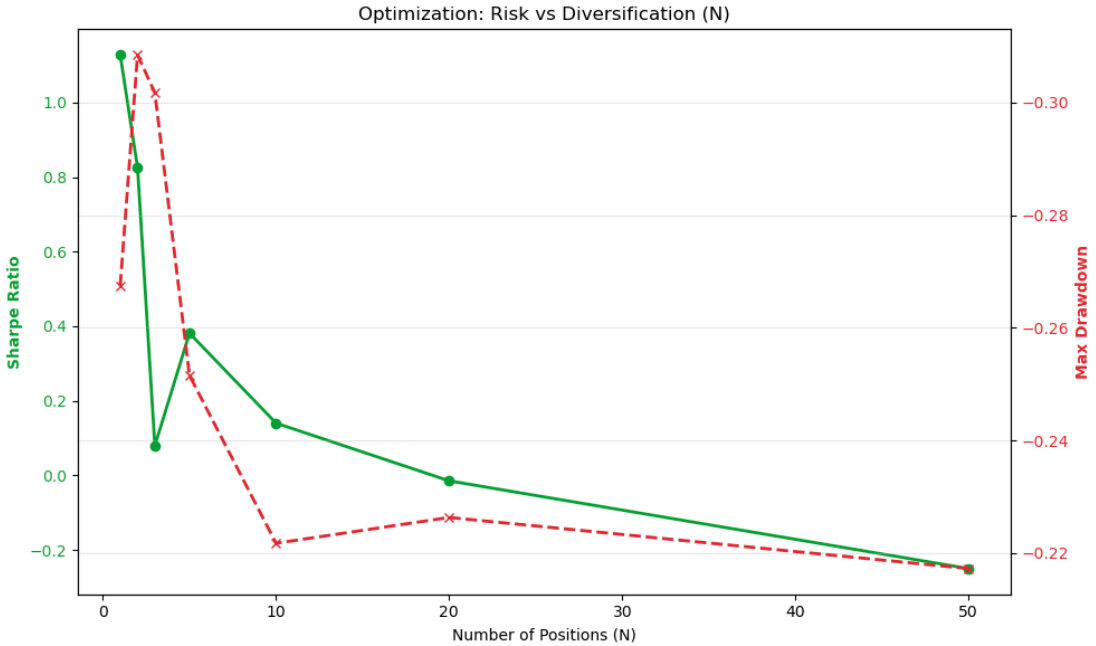




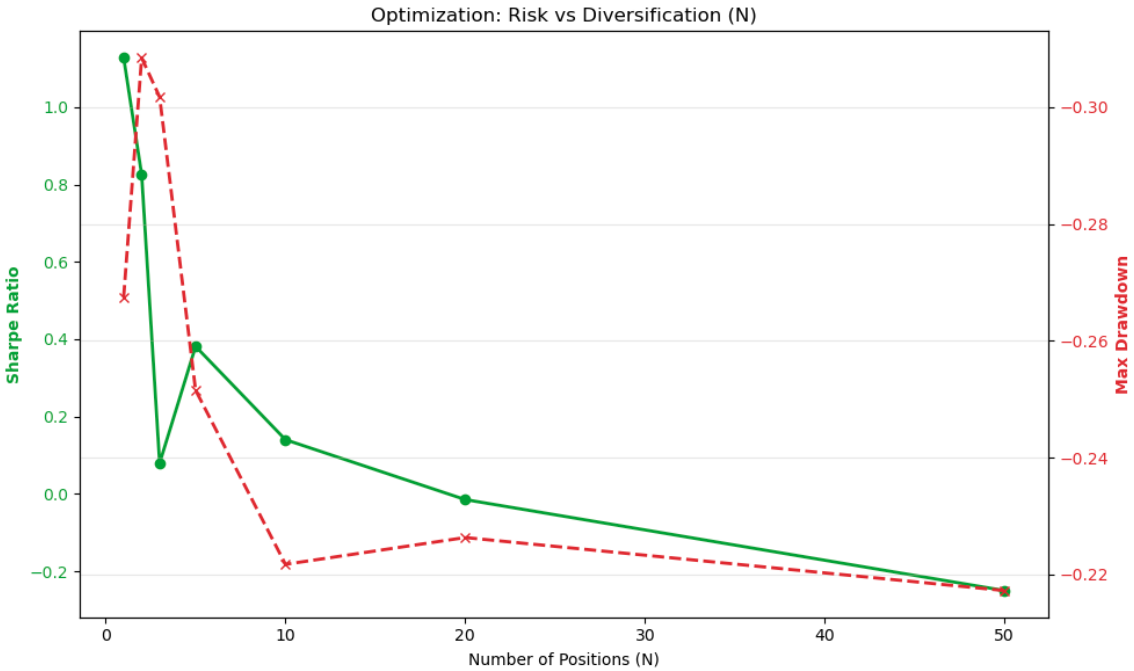


1/27/2026 continued

Equally Weighted



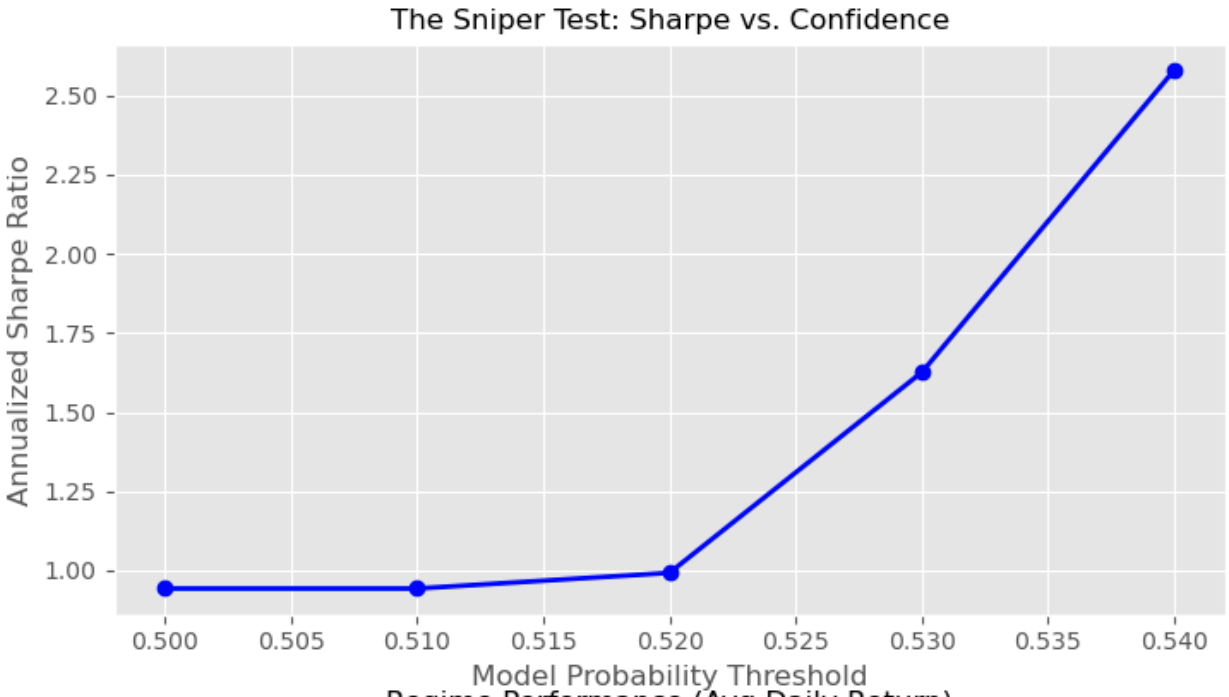
Inverse Volatitlity

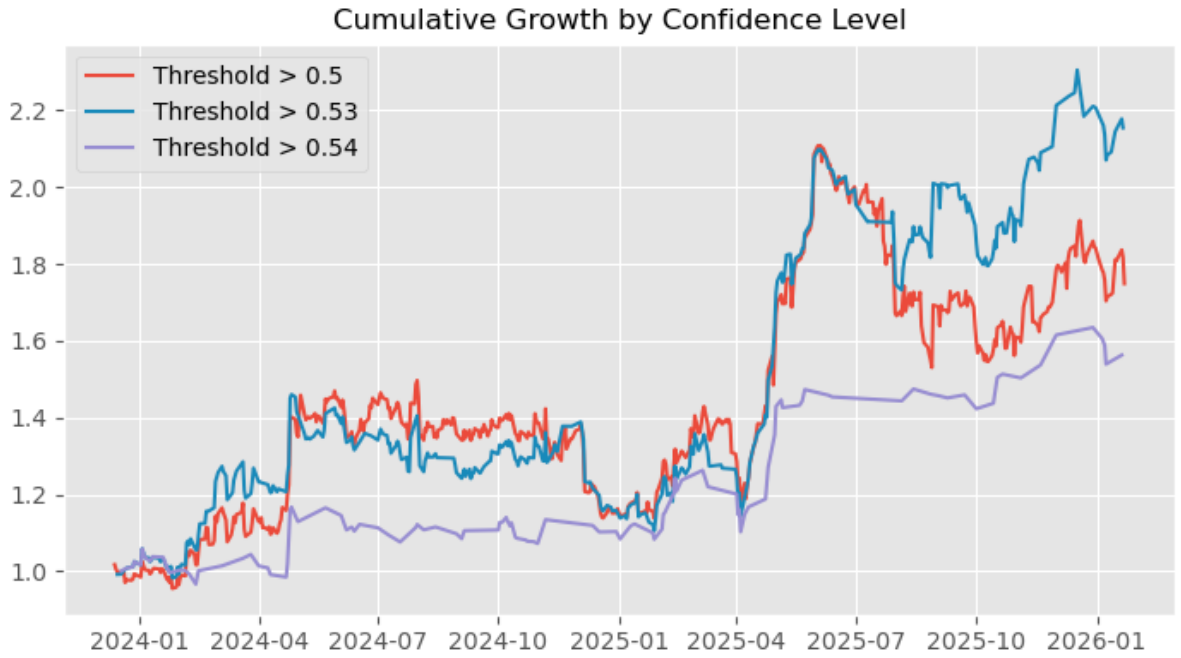


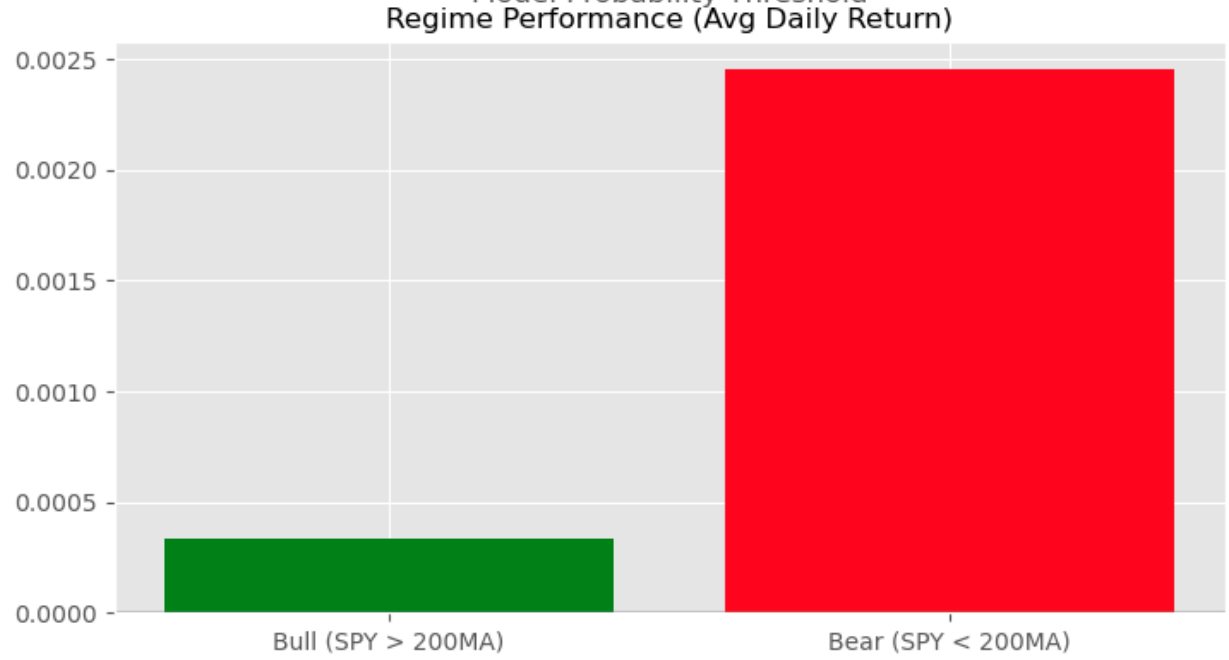
It seems like N=1 is the best strategy. We are not picking the same stock (eg NVDA) every day (results @ right)

```
(Pdb) info_1
           date ticker  pred_ret
0    2023-12-13    PFE  0.526039
1    2023-12-14   TMUS  0.529282
2    2023-12-15    CPT  0.539035
3    2023-12-18     BA  0.540954
4    2023-12-19   STLA  0.527790
..          ...    ...       ...
522  2026-01-15    NOC  0.535625
523  2026-01-16    APD  0.529352
524  2026-01-20    APH  0.540982
525  2026-01-21   CASY  0.530730
526  2026-01-22    MKC  0.526039

[527 rows x 3 columns]
(Pdb) len(info1.ticker.unique())
*** NameError: name 'info1' is not defined
(Pdb) len(info_1.ticker.unique())
321
```







Failure Analysis: Top 5 Blowups

	Ticker	Prob	Return
2024-12-06	GWRE	0.5356663638849297	-0.08209789855639732
2024-08-02	EVR	0.5379709914372507	-0.06598273525607733
2025-07-31	HWM	0.5399180074668133	-0.06012133225657 23
2024-03-21	FDS	0.5322747939477998	-0.057940140923526305
2024-11-07	CRBG	0.5372449452155292	-0.057348500456403516

1/29/2026

- Goal: Organize data for more robust analysis of model
  - Put Universe into class
    - Identifying filters and characteristics in metadata
    - Saves to folder
  - In single class, train and test model
    - Inputs:
      - Universe Strategy (Which pre-defined universe to use)
      - Years to train on
      - Train/test split
    - Methods:
      - Train()
      - Test()
      - Visualization options for the test
  - Once this is done, add in method walk-forward analysis
    - Test how different strategies perform